

ORACLE

Cloud Operations for Exadata Database Service on Exascale Infrastructure

Provisioning Operations Scenarios

Xxx | xxx

Office of CTO | EMEA Technology Engineering
November 6th, 2025

Agenda

- Introduction
- Provisioning Operations Scenarios
 - Provision Storage Vault
 - Provision VM Cluster Networks
 - Provision VM Clusters
 - Provision Oracle Homes
 - Provision Container Databases (CDBs)
 - Provision Pluggable Databases (PDBs)
 - Provision Data Guard Association



Introduction

Introduction



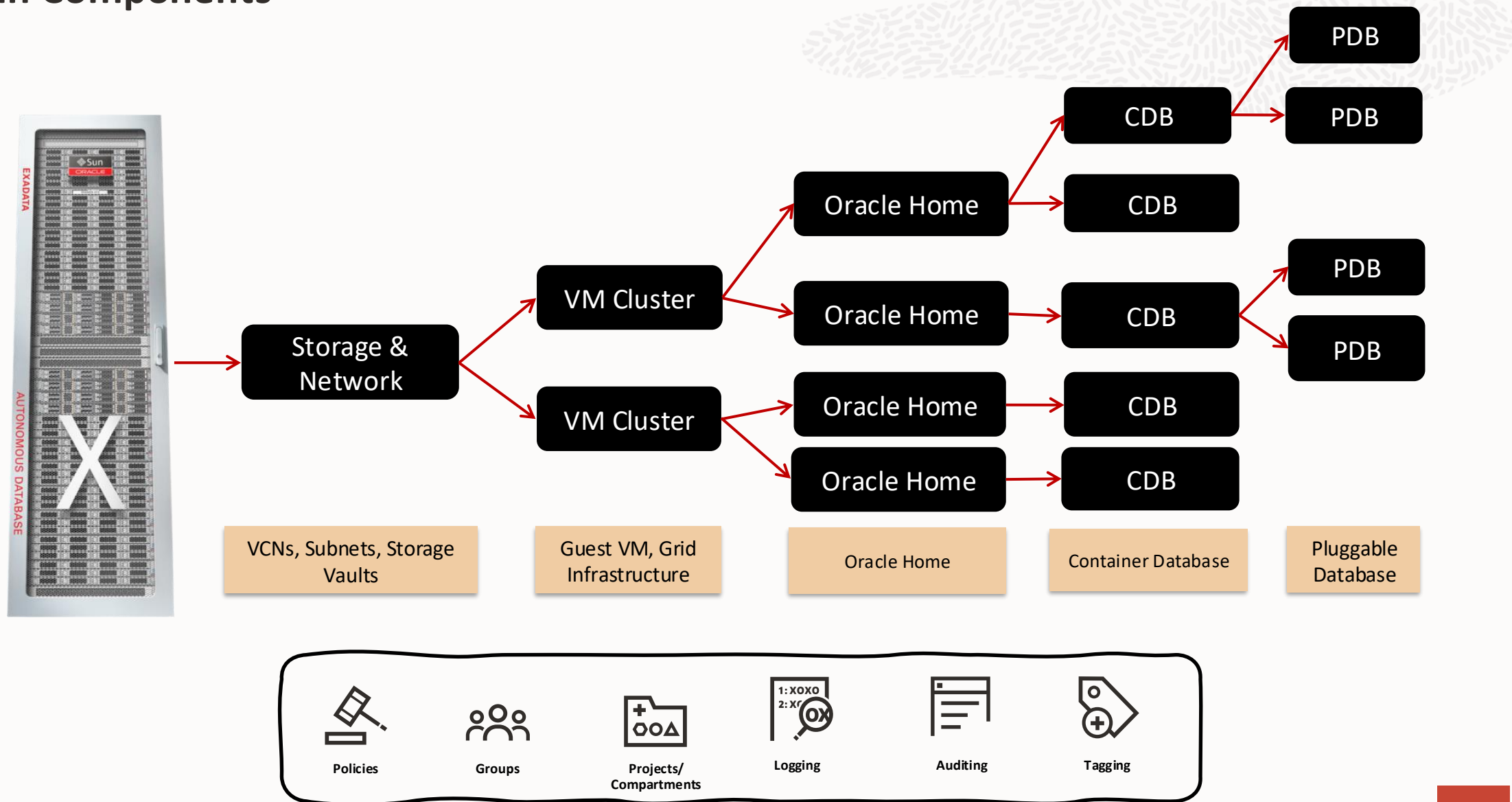
This Workshop provides information about Exadata Database Service on Exascale Infrastructure and its different components, typical operations, and maintenance activities to be performed in the platform by different automation practices

We'll cover different aspects of the infrastructure in terms of service architecture, management interfaces, the information needed to deploy a VC, OH, DBs, DG, scaling and patching

The code in this Workshop is just sample code and can be incomplete.

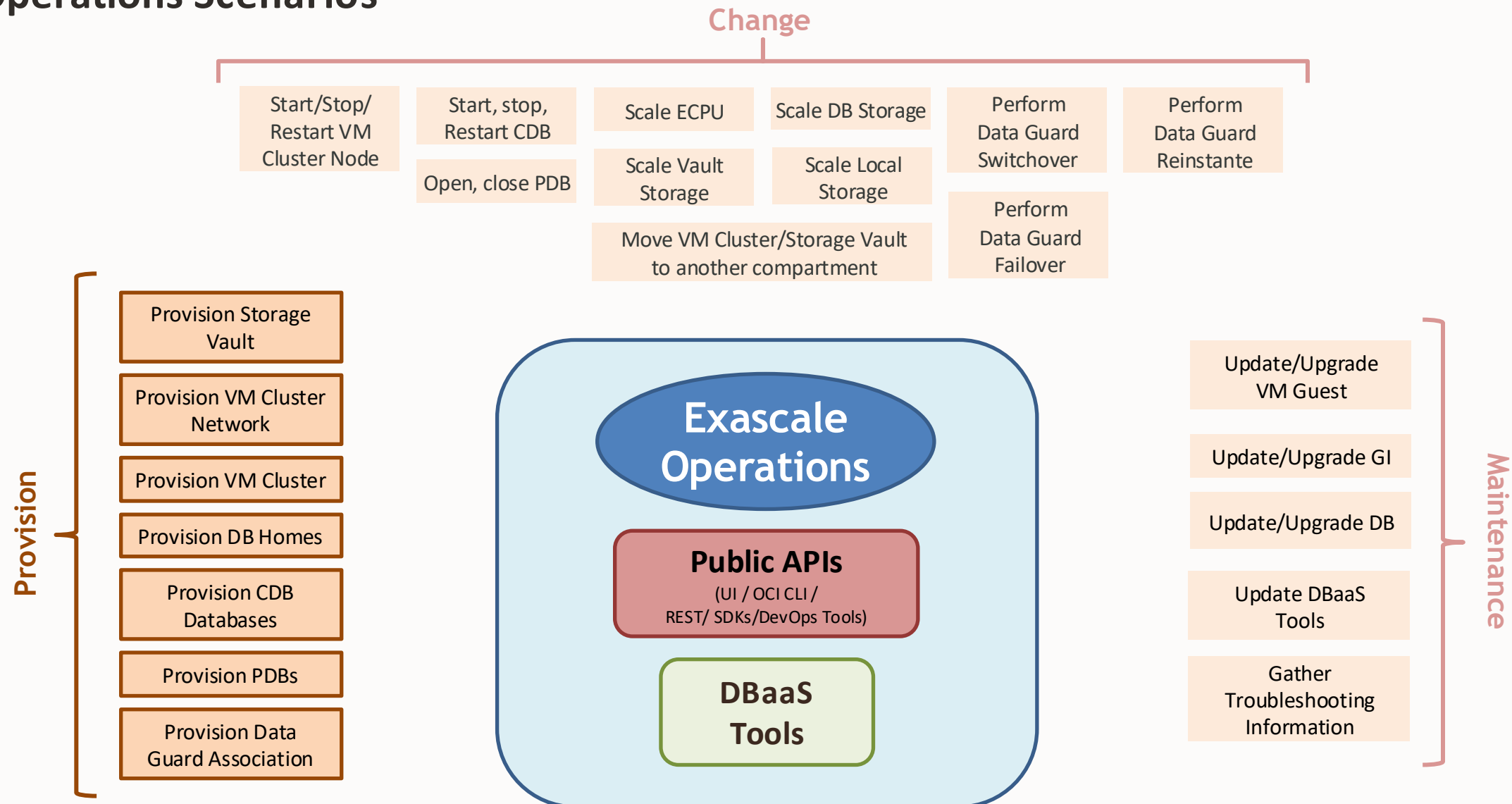
To make the sample code work in your Oracle Cloud tenancy, please replace the values for any parameters whose current values do not fit your use case and check the official documentation to include missing parameters.

Main Components



Provision Operations

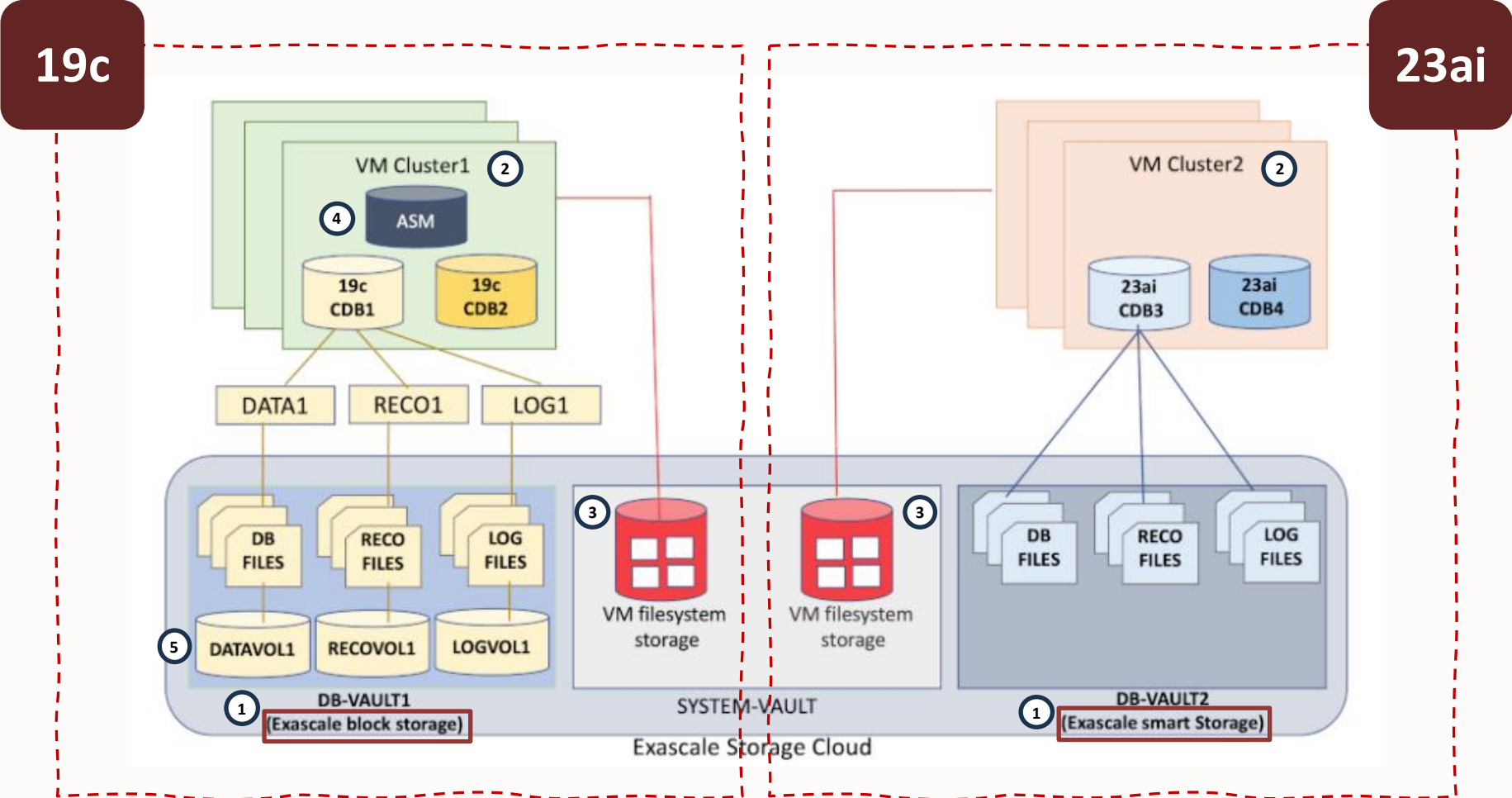
Operations Scenarios



Provision Storage Vault

Exascale Storage Vault

An Exascale vault is a logical storage container that uses the physical resources provided by Exascale storage pools.



OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Cluster → Create VM Cluster → Exascale Database Storage Vault page → **Create new vault**

Delete

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → Exascale Storage Vaults → Select Actions (3 dots) on the Vault → **Delete**

Command Line Interface (OCI CLI)

Create an Exascale Storage Vault

```
oci db exascale-db-storage-vault create \  
  --compartment-id <compartment_ocid> \  
  --availability-domain <av_dom_ocid> [OPTIONS]
```

Delete an Exascale Storage Vault

```
oci db exascale-db-storage-vault delete \  
  --exascale-db-storage-vault-id <xs_db_st_ocid> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_exascale_db_storage_vault"  
  "test_exascale_db_storage_vault" {  
    availability_domain =  
var.exascale_db_storage_vault_availability_domain  
    compartment_id = var.compartment_id  
    display_name =  
var.exascale_db_storage_vault_display_name  
    high_capacity_database_storage {  
      total_size_in_gbs =  
var.exascale_db_storage_vault_high_capacity_database_storage  
_total_size_in_gbs  
    }  
  }  
  ...  
}
```

OCI Control Plane Interfaces

Python SDK

Create an Exascale Storage Vault

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_exascale_db_storage_vault_response =
database_client.create_exascale_db_storage_vault(
    create_exascale_db_storage_vault_details =
oci.database.models.CreateExascaleDbStorageVaultDetails(
    compartment_id="ocidl.test.oc1.<comp-ocid>",
    availability_domain="EXAMPLE-availabilityDomain-Value",
    high_capacity_database_storage =
oci.database.models.ExascaleDbStorageInputDetails(
    total_size_in_gbs=818),
    exadata_infrastructure_id= "
ocidl.test.oc1.<exadataInfrastructureId>",
    cluster_placement_group_id =
"ocidl.test.oc1.<clusterPlacementGroupId>",
    ...),
    ...)
print(create_exascale_db_storage_vault_response.data)
```

Delete an Exascale Storage Vault

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

delete_exascale_db_storage_vault_response =
database_client.delete_exascale_db_storage_vault(
    exascale_db_storage_vault_id =
"ocidl.test.oc1.<exascaleDbStorageVaultId>",
    if_match="EXAMPLE-ifMatch-Value",
    opc_request_id="TFCY3JTBZQF8OV32QQEO<unique ID>")
print(delete_exascale_db_storage_vault_response.headers)
```

OCI APIs

Create an Exascale Storage Vault

```
POST /20160918/exascaleDbStorageVaults
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "displayName" : "exascaleDbStorageVault",
  "compartmentId" : "ocidl.tenancy.oc1.<unique_ID>",
  "availabilityDomain" : "OXET:PHX-AD-1",
  "highCapacityDatabaseStorage": {
    "capacitySizeInGbs": 256
  },
  "additionalFlashCache": 20
}
```

Delete an Exascale Storage Vault

```
DELETE /20160918/exascaleDbStorageVaults/{exascaleDbStorageVaultId}
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

Provision VM Cluster Networks

Provision VM Cluster Network

Requirements

- A **VCN** in the region
- At least **two subnets** in the VCN:
 - Client subnet
 - Backup subnet
- Choose which method of DNS name resolution you will use

Recommendations

- Use regional subnets (span all availability domains in the region)
- Custom route tables for each subnet
- Create security rules to control traffic to and from the client network and backup network of the VM Cluster nodes

Options

- Option 1: Public Client Subnet with Internet Gateway
- Option 2: Private Subnets

Requirements for IP Address Space

- IP addresses must not overlap
- Client subnet:
 - Requires **4** IP addresses **per node**
 - **3** addresses are reserved for SCANs
- Backup subnet
 - Requires **3** IP addresses **per node**
- The Networking service reserves **3** IP addresses in **each subnet**
- To plan for future growth, add addresses that you expect to require as you scale up your VM Cluster, not only the number of VMs you plan to provision for immediate need

How can you create network components?

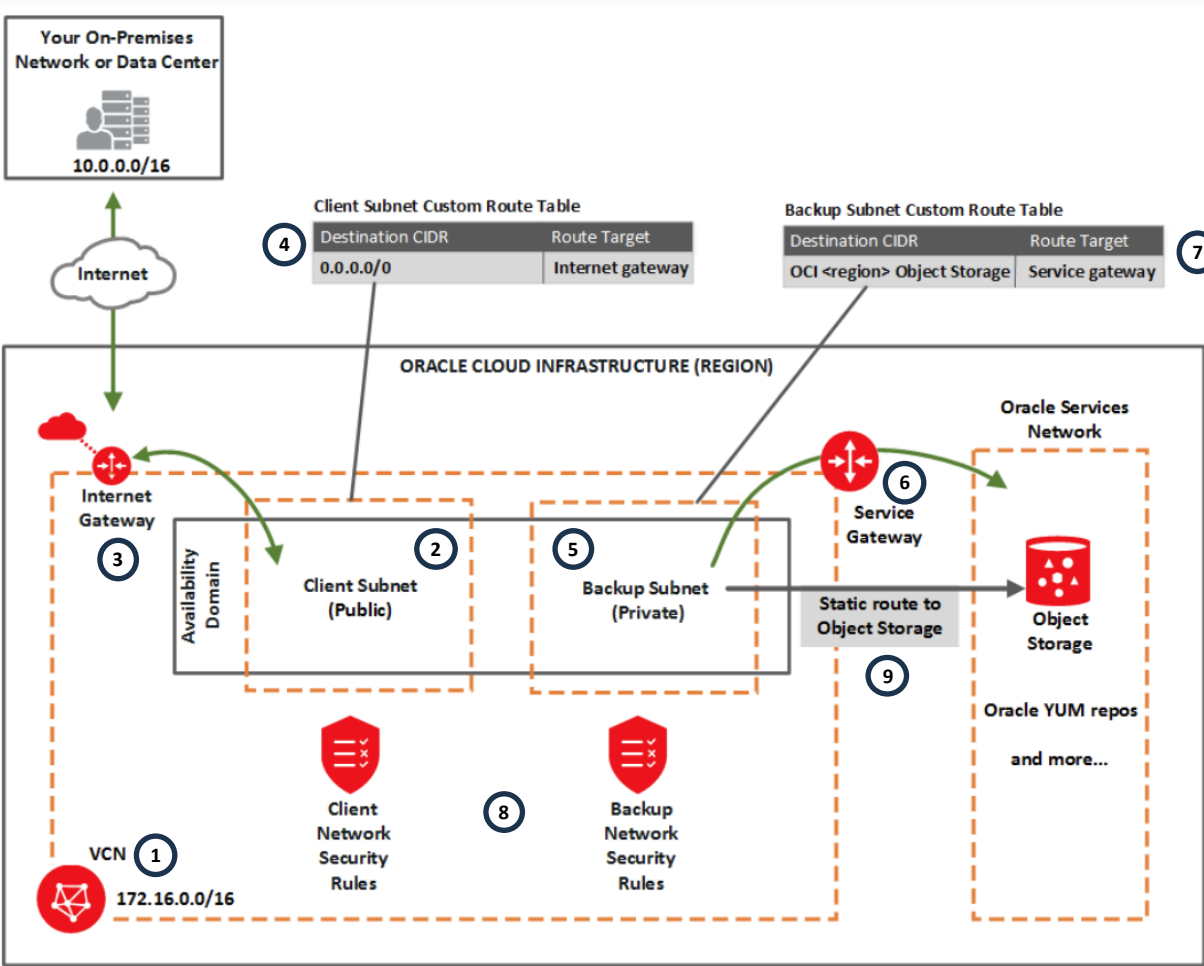
You can create network components by using:

- UI, OCI-CLI, REST API, SDK, IaC

During deployment of VM clusters on Oracle Exadata Database Service on Exascale Infrastructure, you can choose to deploy a default network during VN Cluster creation.

Network Setup

Option 1: Public Client Subnet with Internet Gateway



6 ONLY OBJECT STORAGE ACCESS

7 You configure the backup subnet to use the service gateway for access only to Object Storage.

8

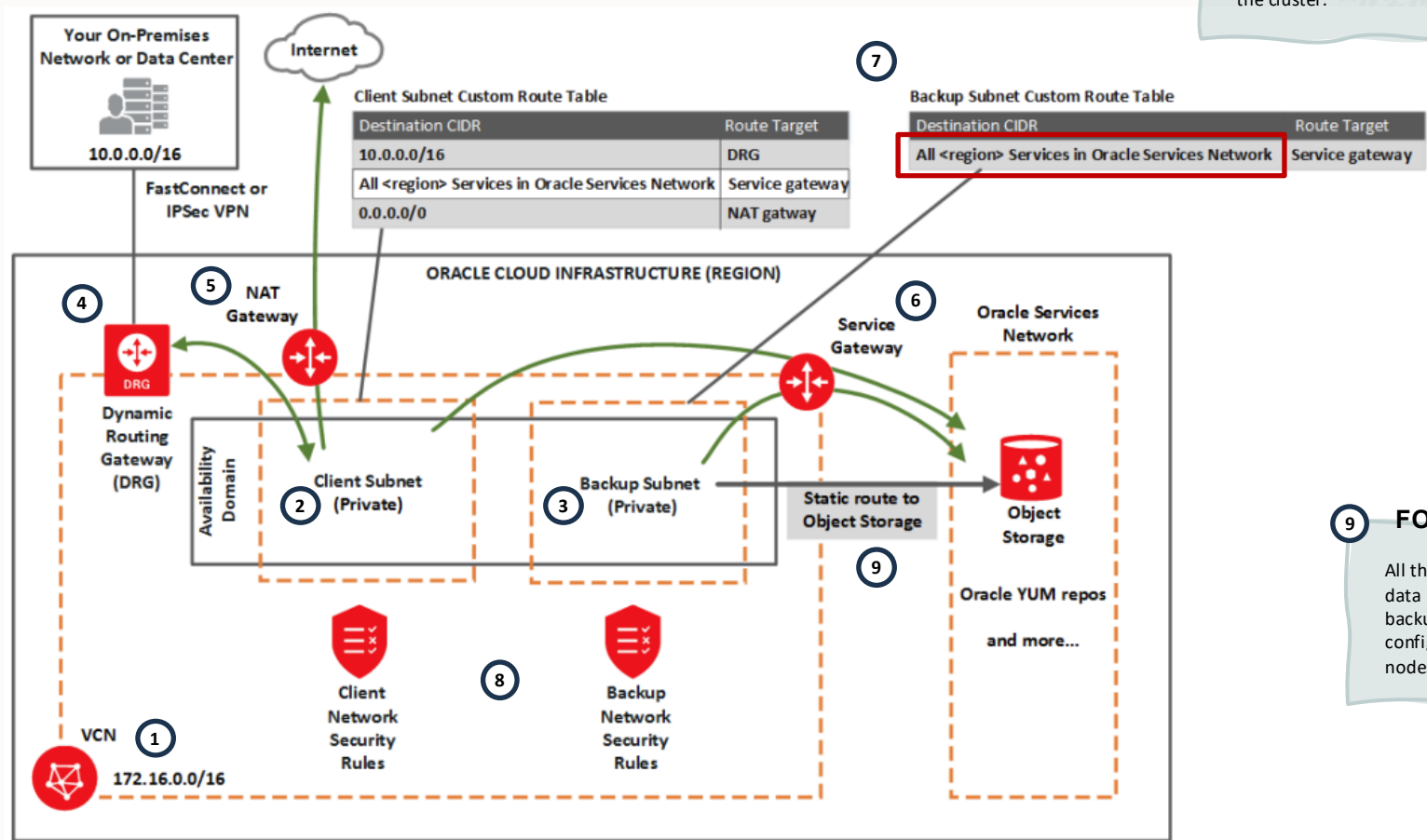
9 FOR CUSTOMIZED BACKUPS

All the traffic is, by default, routed through the data network. To route backup traffic to the backup interface (BONDETH1), you need to configure a static route on each of the compute nodes in the cluster.



Network Setup

Option 2: Private Subnets



6 OBJ. STORAGE AND YUM REPOS ACCESS

- 7 All the traffic is, by default, routed through the data network. To route backup traffic to the backup interface (BONDETH1), you need to configure a static route on each of the compute nodes in the cluster.
- 8

9 FOR CUSTOMIZED BACKUPS

All the traffic is, by default, routed through the data network. To route backup traffic to the backup interface (BONDETH1), you need to configure a static route on each of the compute nodes in the cluster.



Network Setup

Security Rules for the Oracle Exadata Database Service on Exascale Infrastructure



Ingress Rule#	Description	Stateless	Source Type	Source CIDR	IP Protocol	Source Port	Destination Port	Type	Code
General #1	Allows SSH traffic from anywhere	No	CIDR	0.0.0.0/0	SSH	All	22		
General #2	Allows Path MTU Discovery fragmentation messages	No	CIDR	0.0.0.0/0	ICMP			3	4
General #3	Allows connectivity error messages within the VCN	No	CIDR	Your VCN's CIDR	ICMP			3	All
Client #1	Allows ONS and FAN traffic from within the client subnet	No	CIDR	Client subnet's CIDR	TCP	All	6200		
Client #2	Allows SQL*NET traffic from within the client subnet	No	CIDR	Client subnet's CIDR	TCP	All	1521		
Client #3	Allows Patching Traffic from Within the Client Subnet	No	CIDR	Client subnet's CIDR	TCP	All	7085		

Egress Rule#	Description	Stateless	Destination Type	Destination CIDR	IP Protocol	Source Port	Destination Port	Destination Service
General #1	Allows all egress traffic	No	CIDR	0.0.0.0/0	All			
Client #1	Allows all TCP traffic inside the client subnet	No	CIDR	0.0.0.0/0	TCP	All	22	
Client #2	Allows all egress traffic (allows connections to the Oracle YUM repos)	No	CIDR	0.0.0.0/0	All			
Backup #1	Allows access to Object Storage	No	Service		TCP	All	443 (HTTPS)	OCI <region> Object Storage All <region> Services (YUM repo access)



OCI Control Plane Interfaces

OCI Console

Create

Menu → Networking → Virtual Cloud Network → **Create VCN or Actions (Start VCN Wizard)**

Delete

Menu → Networking → Virtual Cloud Network → Select Actions (3 dots) on the VCN → **Delete**

Command Line Interface (OCI CLI)

Create VCN & Subnet

```
oci network vcn create --compartment-id <compartment_ocid> \
  --cidr-block <cidr_block> [OPTIONS]

oci network subnet create --compartment-id <compartment_ocid> \
  --vcn-id <vcn_id> [OPTIONS]
```

Delete VCN & Subnet

```
oci network vcn delete --vcn-id <vcn-id> [OPTIONS]

oci network subnet delete --subnet-id <subnet-id> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_core_vcn" "test_vcn" {
  compartment_id = var.compartment_id
  cidr_block     = var.vcn_cidr_block
  display_name   = var.vcn_display_name
  dns_label      = var.vcn_dns_label
  ...
}

resource "oci_core_subnet" "test_subnet" {
  compartment_id = var.compartment_id
  vcn_id         = oci_core_vcn.test_vcn.id
  ipv4cidr_blocks = var.subnet_ipv4cidr_blocks
  route_table_id =
oci_core_route_table.test_route_table.id
  security_list_ids = var.subnet_security_list_ids
  ...
}
```

OCI Control Plane Interfaces

Python SDK

Create VCN

```
import oci
config = oci.config.from_file()
core_client = oci.core.VirtualNetworkClient(config)

create_vcn_response = core_client.create_vcn(
    create_vcn_details=oci.core.models.CreateVcnDetails(
        compartment_id="ocidl.test.oc1.<compId>",
        cidr_block="EXAMPLE-cidrBlock-Value",
        cidr_blocks=["EXAMPLE--Value"],
        display_name="EXAMPLE-displayName-Value",
        dns_label="EXAMPLE-dnsLabel-Value",
        ...),
    ...)
print(create_vcn_response.data)
```

Delete VCN

```
import oci
config = oci.config.from_file()
core_client = oci.core.VirtualNetworkClient(config)

delete_vcn_response = core_client.delete_vcn(
    vcn_id="ocidl.test.oc1.<unique_ID>EXAMPLE-vcnId-Value",
    if_match="EXAMPLE-ifMatch-Value")
print(delete_vcn_response.headers)
```

OCI APIs

Create a VCN

```
POST /20160918/vcns
Host: iaas.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "cidrBlock" : "10.0.0.0/16",
  "compartmentId" : "ocidl.compartment.oc1.<unique_ID>",
  "displayName" : "MyVcn"
}
```

Delete VCN

```
DELETE /20160918/vcns/{vcnId}
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

OCI Control Plane Interfaces

Python SDK

Create Subnet

```
import oci
config = oci.config.from_file()
core_client = oci.core.VirtualNetworkClient(config)

create_subnet_response = core_client.create_subnet(
    create_subnet_details=oci.core.models.CreateSubnetDetails(
        compartment_id="ocidl.test.oc1.<compID>",
        vcn_id="ocidl.test.oc1.<vcnId>",
        cidr_block="EXAMPLE-cidrBlock-Value",
        ...),
    ...)
print(create_subnet_response.data)
```

Delete Subnet

```
import oci
config = oci.config.from_file()
core_client = oci.core.VirtualNetworkClient(config)

delete_subnet_response = core_client.delete_subnet(
    subnet_id="ocidl.test.oc1.<subnetId>",
    if_match="EXAMPLE-ifMatch-Value")
print(delete_subnet_response.headers)
```

OCI APIs

Create Subnet

```
POST /20160918/subnets
Host: iaas.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "displayName" : "MySubnet",
  "cidrBlock" : "10.0.2.0/24",
  "vcnId" : "ocidl.vcn.oc1.phx.<unique_ID>",
  ...
}
```

Delete Subnet

```
DELETE /20160918/subnets/{subnetId}
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

Provision VM Cluster

Provision a VM Cluster

*A **VM Cluster** is a group of virtual machines with shared compute, networking, and Exascale storage resources, configured to act as a unified environment for hosting Oracle Database instances.*

Requirements

- Storage Vault
- VCN and Subnets

Considerations

- By default, uses Enterprise Edition - Extreme Performance
- CDB created with optional first pluggable database
- No parallel creation is allowed
- Database would be created in all the nodes of the VM Cluster
- SGA_TARGET is set by the automation:
 - VM memory ≤ 60 GB $\rightarrow$ SGA_TARGET = 3800 MB
 - VM memory ≥ 60 GB $\rightarrow$ SGA_TARGET = 7600 MB.
- All DBs are configured using USE_LARGE_PAGES=ONLY
- Default (and recommended) hugepages space in the OS is equal to half of the total amount of RAM memory

Versions

- Oracle AI Database 26ai
 - Exascale Smart Storage
 - Direct vault access (no ASM)
 - No manual storage allocation needed
- Oracle Database 19c
 - Exascale Block Storage
 - Uses ASM (DATA, RECO, LOG)
 - Manual storage allocation required

How can you create a VM Cluster?

You can create a VM cluster by using:

- UI, OCI-CLI, REST API, SDK, IaC

Why create additional VM Clusters?

Security separation

- Provide separation between Dev and Test environments, or between multiple production databases
- VMs can be used to address isolation for data security requirements including PCI data and HIPAA data

Maintenance

- Isolate group of database and minimize the potential downtime for GI patching

Blast radius

- Virtualization limits the blast radius in case of system failure

Resource management

- Ensures that database receives the resources it requires
- Minimizes the 'noisy neighbor' issue
- Possible licensing limitation approach

Administrative separation

- An organization that have multiple system administrator teams

- Each VM brings additional administrative overhead, including installation, patching, security, and management
- Excessive isolation, such as the “one DB per VM” approach, results in virtual sprawl.
- Oracle recommends a more judicious use of VMs, including deployment of small numbers of VMs per physical machine, with each VM hosting multiple CDBs, and/or PDBs

OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → **Create VM Cluster**

Delete

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Select Actions (3 dots) on the VM Cluster → **Terminate**

Command Line Interface (OCI CLI)

Create VM Cluster

```
oci db exadb-vm-cluster create --compartment-id <comp_ocid> \
  --availability-domain <av_dom_ocid> \
  --display-name <vm_name> [OPTIONS]
```

Delete VM Cluster

```
oci db exadb-vm-cluster delete \
  --exadb-vm-cluster-id <vm_cluster_ocid> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_exadb_vm_cluster" "test_exadb_vm_cl"
{
  availability_domain =
var.exadb_vm_cluster_availability_domain
  backup_subnet_id = oci_core_subnet.test_subnet.id
  compartment_id = var.compartment_id
  display_name = var.exadb_vm_cluster_display_name
  exascale_db_storage_vault_id =
oci_database_exascale_db_storage_vault.test_exascale_db_stor
age_vault.id
  grid_image_id = oci_core_image.test_image.id
  hostname = var.exadb_vm_cluster_hostname
  shape = var.exadb_vm_cluster_shape
  subnet_id = oci_core_subnet.test_subnet.id
  ...
}
```

OCI Control Plane Interfaces

Python SDK

Create VM cluster

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_exadb_vm_cluster_response =
database_client.create_exadb_vm_cluster(
    create_exadb_vm_cluster_details =
oci.database.models.CreateExadbVmClusterDetails(
    compartment_id="ocidl.test.oc1.<compartmentId>",
    availability_domain="EXAMPLE-availabilityDomain-Value",
    subnet_id="ocidl.test.oc1.<subnetId>",
    hostname="EXAMPLE-hostname-Value",
    node_count=508,
    total_e_cpu_count=336,
    grid_image_id="ocidl.test.oc1.GIImageId",
    shape_attribute="SMART_STORAGE",
    ...),
    ...)
print(create_exadb_vm_cluster_response.data)
```

Delete VM cluster

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

delete_exadb_vm_cluster_response =
database_client.delete_exadb_vm_cluster(
    exadb_vm_cluster_id="ocidl.test.oc1.<exadbVmClId>",
    if_match="EXAMPLE-ifMatch-Value",
    opc_request_id="CYP0Y4Q4PCL2ZWU68M7Q<unique_ID>")
print(delete_exadb_vm_cluster_response.headers)
```

OCI APIs

Create VM cluster

```
POST /20160918/exadbVmClusters
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "displayName" : "exadbVmCluster",
  "compartmentId" : "ocidl.tenancy.oc1.<unique_ID>",
  "availabilityDomain" : "OXET:PHX-AD-1",
  "hostname" : "apollo",
  "scanListenerPortTcp" : "1521",
  "scanListenerPortTcpSsl" : "2484",
  "giVersion" : "19.0.0.0",
  "totalECpuCount" : 10,
  "enabledECpuCount" : 2,
  "exadbStorageVaultId":
"ocidl.exadbStorageVault.oc1.<unique_ID>",
  "nodeCount": 2,
  ...
}
```

Delete VM cluster

```
DELETE /20160918/exadbVmClusters/{exadbVmClusterId}
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```


Provision Oracle Database Homes

Provision an Oracle Database Home

*A **Database Home** is a directory location on the Exadata database virtual machines that contains Oracle Database software binary files.*

How can you create a Database Home?

You can add Oracle Database homes to an existing VM cluster by using:

- UI, OCI-CLI, REST API, SDK, IaC
- dbaascli

Options:

- Only deploy a Database Home
- Create a Database Home at Database creation

➤ With both methods we can just select the Database version or select a Custom Database Software image

➤ If we want other PSU/RU we need to download it previously using dbasstools

➤ If you need to install custom one-offs, you can do it with traditional methods after Oracle Home deployment, but the recommended approach is to include them in a Custom Database Software image.



Do not download RDBMS binaries from the internet or MOS to install it, as you would be outside automation, probably you would lack of some important patches or one-offs specific for Exadata Cloud and it could be a source of problems

OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Database homes → **Create database database**

Delete

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Database homes → Select Actions (3 dots) on the database home → **Delete**

Command Line Interface (OCI CLI)

Create database

```
oci db db-home create --vm-cluster-id <vm-cl-ocid> \
  --database-software-image-id <image_ocid> [OPTIONS]
```

Delete database

```
oci db db-home delete --db-home-id <dbhome_ocid> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_db_home" "test_db_home" {
  database_software_image_id =
oci_database_database_software_image.test_database_software_
image.id
  db_system_id = oci_database_db_system.test_db_system.id
  defined_tags = var.db_home_defined_tags
  display_name = var.db_home_display_name
  vm_cluster_id =
oci_database_vm_cluster.test_vm_cluster.id
  ...
}
```

OCI Control Plane Interfaces

Python SDK

Create a database home

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_db_home_response = database_client.create_db_home(
    create_db_home_with_db_system_id_details =
oci.database.models.CreateDbHomeWithDbSystemIdFromBackupDetails
(
    source="DB_BACKUP",
    db_system_id="ocidl.test.oc1..<dbSystemId>",
    database =
oci.database.models.CreateDatabaseFromBackupDetails(
        backup_id="ocidl.test.oc1..<backupId>",
        ...),
    display_name="EXAMPLE-displayName-Value",
    database_software_image_id =
"ocidl.test.oc1..<DBSoftImaId>",
    ...),
    ...)
print(create_db_home_response.data)
```

Delete a database home

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

delete_db_home_response = database_client.delete_db_home(
    db_home_id="ocidl.test.oc1..<dbHomeId>",
    if_match="EXAMPLE-ifMatch-Value",
    perform_final_backup=False)
print(delete_db_home_response.headers)
```

OCI APIs

Create a database home

```
POST /20160918/dbHomes
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
    "dbSystemId" : "ocidl.dbsystem.oc1.phx.<unique_ID>",
    "displayName" : "createdDbHome",
    "source": "NONE",
    "dbVersion" : "19.3.0.0",
}
```

Delete a database home

```
DELETE /20160918/dbHomes/<dbHomeId>
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

Local VM Command-Line Interfaces - dbaascli

Database software images are resources within your tenancy that you create prior to provisioning or patching an Oracle Exadata Database Service on Exascale Infrastructure instance, Database Home, or database.

List locally available DB/GI images

```
$ dbaascli cswLib listLocal [--product DATABASE | GRID]
```

Download/update an image

```
$ dbaascli cswLib download [--imageTag <value> | --version <value> ] [--product DATABASE | GRID]
```

List available DB/GI images in CPS

```
$ dbaascli cswLib showImages [--product DATABASE | GRID] [--jsonFormat [--jsonFilePath <value>]]
```

Delete a local image

```
$ dbaascli cswLib deleteLocal --imageTag <value>
```

¹... Not all parameters listed. Please check the command definition

Local VM Command-Line Interfaces - dbaascli

Create a new database home

```
$ dbaascli dbHome create --version <value> [--oracleHomeName <value>] [--oracleHome <value>] [--imageTag <value>]  
  [--enableUnifiedAuditing] [--executePrereqs] [--imageFilePath <value>] [--waitForCompletion <value>]
```

Delete the given database home

```
$ dbaascli dbHome delete {--oracleHome <value> | --oracleHomeName <value>} [--resume [--sessionID <value>]]  
  [--waitForCompletion <value>]
```

Display details about a specific database home

```
$ dbaascli dbHome getDetails {--oracleHomeName <value> | --oracleHome <value>}
```

Display details of all the database homes

```
$ dbaascli system getDBHomes
```

Display details of databases in specific database home

```
$ dbaascli dbHome getDatabases {--oracleHomeName <value> | --oracleHome <value>}
```

¹... Not all parameters listed. Please check the command definition

Provision Container Databases

Provision an Oracle Container Database

*A **Container Database (CDB)** is an Oracle database that functions as a container for zero, one, or many pluggable databases (PDBs).*

Requirements

- VM Cluster Configured
- Backup Destinations Created
 - Autonomous Recovery Service or Object Storage bucket
- TDE Encryption Required

Considerations

- By default, uses Enterprise Edition - Extreme Performance
- CDB created with optional first pluggable database
- No parallel creation is allowed
- Database would be created in all the nodes of the VM Cluster
- SGA_TARGET is set by the automation:
 - VM memory <= 60 GB → SGA_TARGET = 3800 MB
 - VM memory >= 60 GB → SGA_TARGET = 7600 MB.
- All DBs are configured using USE_LARGE_PAGES=ONLY
- Default (and recommended) hugepages space in the OS is equal to half of the total amount of RAM memory

Versions

- Oracle AI Database 26ai
 - Exascale Smart Storage
 - Direct vault access (no ASM)
 - No manual storage allocation needed
- Oracle Database 19c
 - Exascale Block Storage
 - Uses ASM (DATA, RECO, LOG)
 - Manual storage allocation required

How can you create a Container Database?

You can create an Oracle Container Database to an existing VM cluster by using:

- UI, OCI-CLI, REST API, SDK, IaC
- dbaascli

OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Container databases → **Create container database**

Delete

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Container databases → Select Actions (3 dots) on the CDB → **Terminate**

Command Line Interface (OCI CLI)

Create database

```
oci db database create --compartment-id <compartment_ocid> \
  --db-home-id <db_home_ocid> \
  --db-name <db_name> [OPTIONS]
```

Delete database

```
oci db database delete --database-id <database_ocid> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_database" "cdb" {
  compartment_id = var.compartment_id
  db_home_id     = var.db_home_id
  display_name   = "<db_display_name>"
  db_name        = "<db_name>"
  ...
}
```

OCI Control Plane Interfaces

Python SDK

Create database from backup

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_database_response = database_client.create_database(
    create_new_database_details =
oci.database.models.CreateDatabaseFromBackup(
    source="DB_BACKUP",
    db_home_id="ocidl.test.oc1.<unique_dbHomeId-Value>",
    db_version="EXAMPLE-dbVersion-Value",
    database =
oci.database.models.CreateDatabaseFromBackupDetails(
    backup_id="ocidl.test.oc1.<unique_bckId-Value>",
    db_unique_name="EXAMPLE-dbUniqueName-Value",
    ...),
    ...),
    ...)
print(create_database_response.data)
```

Delete database

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

delete_database_response = database_client.delete_database(
    database_id="ocidl.test.oc1.<unique_databaseId-Value>",
    if_match="EXAMPLE-ifMatch-Value",
    perform_final_backup=True,
    opc_request_id="XPOVWIIQCSSZX1I7ATSP0<unique_ID>")

print(delete_database_response.headers)
```

OCI APIs

Create a new database

```
POST /20160918/databases
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "dbHomeId" : "ocidl.dbhome.oc1.phx.<unique_ID>",
  "source": "NONE",
  "database" : {
    "adminPassword" : "<password>",
    "dbName" : "myTestDb",
    "dbBackupConfig" : {
      "recoveryWindowInDays" : 30,
      "autoBackupEnabled" : true
    }
  },
  "kmsKeyId" : "ocidl.key.oc1.<unique_ID>",
  "kmsKeyVersionId" : "ocidl.keyversion.oc1.<unique_ID>"
}
```

Delete database

```
DELETE /20160918/databases/<databaseId>
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

Local VM Command-Line Interfaces - dbaascli

Create a new database

```
$ dbaascli database create --dbName <value> {--oracleHome <value> | --oracleHomeName <value>}  
  [--dbUniqueName <value>] [--dbSID <value>] 1...
```

Create a new database from active database

```
$ dbaascli database duplicate --dbName <value> --sourceDBConnectionString <value>  
  {--oracleHome <value> | --oracleHomeName <value>} [--dbSID <value>] [--dbUniqueName <value>] 1...
```

Convert given non-CDB database to PDB

```
$ dbaascli database convertToPDB --dbname <value> {--copyDatafiles [--keepSourceDB] | --backupPrepared}  
  [--cdbName <value>] 1...
```

Create a new database template

```
$ dbaascli database createTemplate --dbname <value> {--templateLocation <value> | --uploadToObjectStorage  
  --objectStorageLoginUser <value> --objectStorageBucketName <value> [--objectStorageUrl <value>]} 1...
```

Delete the given database

```
$ dbaascli database delete --dbname <value> [--deleteArchiveLogs <value>] [--deleteBackups <value>] 1...
```

¹... Not all parameters listed. Please check the command definition

Provision Pluggable Databases

Provision a Pluggable Database

*A **Pluggable Database (PDB)** is a self-contained, portable collection of schemas and objects that operates within a multitenant Container Database (CDB).*

Requirements

- CDB already created
- Oracle Database 19c and later

Considerations

- During PDB creation the parent database (CDB) is in “Updating” state
- You can just create an Oracle PDB at the same time (not parallel creation is allowed)
- PDB would be created in all the nodes of the VM Cluster.
- For non-RAC databases use a PDB service with cardinality=1 and availability in all the cluster nodes

Limitations

- **Discovery:** PDBs created via SQL aren’t shown in OCI Console immediately; OCI syncs them automatically within ~45 minutes.
- **Backups:** Console/API backups occur at the CDB level and include all PDBs; dbaascli can back up individual PDBs.
- **Restores:** Console/API restores are CDB-level; dbaascli allows restoring specific PDBs.
- **Data Guard:** Creating a (PDB) using the console is not supported for databases using Data Guard.

How can you create a Pluggable Database?

You can create a Pluggable Database to an existing Container Database by using:

- UI, OCI-CLI, REST API, SDK, IaC
- dbaascli, SQL (not recommended)

OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Container databases → Select CDB → Pluggable Databases → **Create**

Delete

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Container databases → Select CDB → Pluggable Databases → Select Actions (3 dots) on the PDB → **Delete**

Command Line Interface (OCI CLI)

Create PDB

```
oci db pluggable-database create -pdb-name <pdb_name> \
  --container-database-id <cdb_ocid> [OPTIONS]
```

Delete PDB

```
oci db pluggable-database delete \
  --pluggable-database-id <pdb_ocid> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_pluggable_database" "test_pdb" {
  #Required
  container_database_id = oci_database_database.test_cdb.id
  pdb_name = var.pluggable_database_pdb_name
  ...
}
```

OCI Control Plane Interfaces

Python SDK

Create PDB

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_pluggable_database_response =
database_client.create_pluggable_database(
    create_pluggable_database_details =
oci.database.models.CreatePluggableDatabaseDetails(
    pdb_name="EXAMPLE-pdbName-Value",
    container_database_id = "ocidl.test.ocl..<CDBId>",
    pdb_creation_type_details =
oci.database.models.CreatePluggableDatabaseFromRelocateDetails(
    creation_type="RELOCATE_PDB",
    source_pluggable_database_id="ocidl.test.ocl..<ID>",
    source_container_database_admin_password="AdmPwd",
    dblink_username="EXAMPLE-dblinkUsername-Value",
    dblink_user_password="EXAMPLE-dblinkUserPwd"),
    ...),
    ...)
print(create_pluggable_database_response.data)
```

Delete PDB

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

delete_pluggable_database_response =
database_client.delete_pluggable_database(
    pluggable_database_id = "ocidl.test.ocl..<PDBId>",
    if_match="EXAMPLE-ifMatch-Value",
    opc_request_id="6UDOSRKRXDV7XZ017PIA<unique_ID>")
print(delete_pluggable_database_response.headers)
```

OCI APIs

Create PDB

```
POST /20160918/pluggableDatabases
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
    "pdbName" : "SalesPdb",
    "tdeWalletPassword" : "<TDE_password>"
    "containerDatabaseId" :
"ocidl.database.ocl..aaaaaaaah2sauv373xyfrpcnaed2ptfy67fnspzyda
2hacgdbarki jhffugaa",
    "pdbAdminPassword" : "<password>"
}
```

Delete PDB

```
DELETE /20160918/pluggableDatabases/<pluggableDatabaseId>
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
```

Local VM Command-Line Interfaces - dbaascli

Create a new PDB

```
$ dbaascli pdb create --pdbName <value> --dbName <value> 1...
```

Delete a PDB

```
$ dbaascli pdb delete --dbName <value> {--pdbName <value> | --pdbUID <value>} 1...
```

Clone a PDB from another PDB in the same database

```
$ dbaascli pdb localClone --pdbName <value> --dbName <value> [--targetPDBName <value>] 1...
```

Clone the specified PDB from another database

```
$ dbaascli pdb remoteClone --pdbName <value> --dbName <value> --sourceDBConnectionString <value> 1...
```

Relocate the specified PDB from remote DB into local DB

```
$ dbaascli pdb relocate --pdbName <value> --dbName <value> --sourceDBConnectionString <value> 1...
```

Refresh a specified PDB

```
$ dbaascli pdb refresh --dbname <value> {--pdbName <value> | --pdbUID <value>} [--waitForCompletion <value>]
```

¹... Not all parameters listed. Please check the command definition

Provision Data Guard Association

Provision a Data Guard Association

*A **Data Guard association** is the logical pairing between a primary database and its standby database that defines their relationship, replication mode, and failover/switchover capabilities.*

Requirements

- **Infrastructure:**
 - Two separate Exadata VM Clusters
 - Same VCN and region (or VCN peering configured for cross-region)
 - Standby VM Cluster in a different availability domain (recommended)
- **Database:**
 - Primary and standby databases must have the same version
 - Primary database must exist and be functioning
- **Database Home:**
 - Create a new DB Home on standby instance OR use an existing one
 - Standby DB Home matching all patches matching on the primary
- **Network:**
 - TCP port 1521 open between primary and standby subnets
 - Ingress and egress security rules configured (stateful)
 - Data Guard replication occurs over the client network
- **Encryption (if using customer-managed keys):**
 - Oracle Vault Service must be configured
 - Master encryption key must be created
 - Key version OCID must be available
- **Passwords:**
 - SYS password required for primary database
 - TDE wallet password (if using Oracle-managed encryption)

Provision a Data Guard Association

Configurations & Types

- Physical Standby
- Up to 6 standby databases
- Data Guard Type:
 - Data Guard or Active Data Guard
- Protection Mode:
 - Maximum Availability (Sync) or Maximum Performance (Async)
- Cross-Service Data Guard:
 - Cross-service Oracle Data Guard between Exadata Database Service on Dedicated Infrastructure and Exadata Database Service on Exascale Infrastructure can be configured only with the Oracle AI Database 26ai

Restrictions

- **You can't terminate a primary with an active standby:** delete the standby first or perform a switchover.
- **You can't terminate a VM cluster containing Data Guard enabled databases:** remove the DG association by deleting the standby first.

Supported Operations

- **Switchover:** Reverses the primary and standby database roles.
- **Failover:** Transitions the standby database to the primary role after the primary fails or becomes unreachable.
- **Reinstate:** Reinstates a failed database into the standby role after correcting the cause of failure.
- **PDB creation:** Not supported using the console

How can you create a Data Guard Association?

You can create a Data Guard Association to an existing Container Database by using:

- UI, OCI-CLI, REST API, SDK, IaC
- dbaascli, Data Guard Broker, SQL (not recommended)

OCI Control Plane Interfaces

OCI Console

Create

Menu → Oracle AI Database → Oracle Exadata Database Service on Exascale Infrastructure → VM Clusters → Container databases → Select CDB → Data Guard Associations → **Add standby**

Command Line Interface (OCI CLI)

Create Data Guard Association

```
oci db data-guard-association create from-existing-vm-cluster \  
  --database-admin-password <adm_pwd> \  
  --database-id <db_ocid> \  
  --peer-vm-cluster-id <cluster_ocid> \  
  --protection-mode <protection_mode> \  
  --transport-type <transport_type> [OPTIONS]
```

Terraform - Oracle Cloud Infrastructure Provider

```
resource "oci_database_data_guard_association"  
  "test_dg_assoc" {  
    #Required  
    creation_type = var.data_guard_association_creation_type  
    database_admin_password =  
var.data_guard_association_database_admin_password  
    database_id = oci_database_database.test_database.id  
    delete_standby_db_home_on_delete =  
var.data_guard_association_delete_standby_db_home_on_delete  
    protection_mode =  
var.data_guard_association_protection_mode  
    transport_type =  
var.data_guard_association_transport_type  
    ...  
  }
```

To **terminate** an Oracle Exadata Database Service on Exascale Infrastructure instance Data Guard association, you **delete the standby database**.

OCI Control Plane Interfaces

Python SDK

Create Data Guard Association

```
import oci
config = oci.config.from_file()
database_client = oci.database.DatabaseClient(config)

create_data_guard_association_response =
database_client.create_data_guard_association(
    database_id="ocidl.test.oc1..<DBID>",
    create_data_guard_association_details =
oci.database.models.CreateDataGuardAssociationToExistingDbSystemDetails(
    creation_type="ExistingDbSystem",
    database_admin_password="EXAMPLE-DBAdminPWD",
    protection_mode="MAXIMUM_AVAILABILITY",
    transport_type="SYNC",
    database_software_image_id =
"ocidl.test.oc1..<DBSoftImageID>",
    source_encryption_key_location_details =
oci.database.models.ExternalHsmEncryptionDetails(
        provider_type="EXTERNAL",
        hsm_password="EXAMPLE-hsmPassword-Value"),
    is_active_data_guard_enabled=False,
    peer_db_unique_name="EXAMPLE-peerDbUniqueName-Value",
    peer_sid_prefix="EXAMPLE-peerSidPrefix-Value",
    peer_db_system_id="ocidl.test.oc1..<peerDbSystemId>",
    peer_db_home_id="ocidl.test.oc1..<peerDbHomeId>"),
    opc_retry_token="EXAMPLE-opcRetryToken-Value")
print(create_data_guard_association_response.data)
```

OCI APIs

Create Data Guard Association

```
POST /20160918/databases/<databaseId>/dataguardassociations
Host: database.us-phoenix-1.oraclecloud.com
<authorization and other headers>
{
  "databaseAdminPassword" : "<password>"
  "protectionMode" : "MAXIMUM_PERFORMANCE"
  "creationType" : "ExistingDbSystem"
  "transportType" : "ASYNC",
  "peerDbSystemId" : "ocidl.dbsystem.oc1.phx.<unique_ID>"
}
```

To **terminate** an Oracle Exadata Database Service on Exascale Infrastructure instance Data Guard association, you **delete the standby database**.

Local VM Command-Line Interfaces - dbaascli

Configure a Standby in an existing Data Guard setup

```
$ dbaascli dataguard configureStandby --dbname <value> --standbyDBUniqueName <value> 1...
```

Configure an Oracle database as a Primary database

```
$ dbaascli dataguard prepareForStandby --dbname <value> --standbyDBUniqueName <value> 1...
```

Export Primary wallet for Standby creation

```
$ dbaascli dataguard prepareStandbyBlob --dbname <value> --blobLocation <value>
```

Enable multiple Standby support on DG setup

```
$ dbaascli dataguard enableMultipleStandbySupport --dbname <value> [--executePrereqs] [--primaryDBOCID <value>] 1...
```

Register Standby

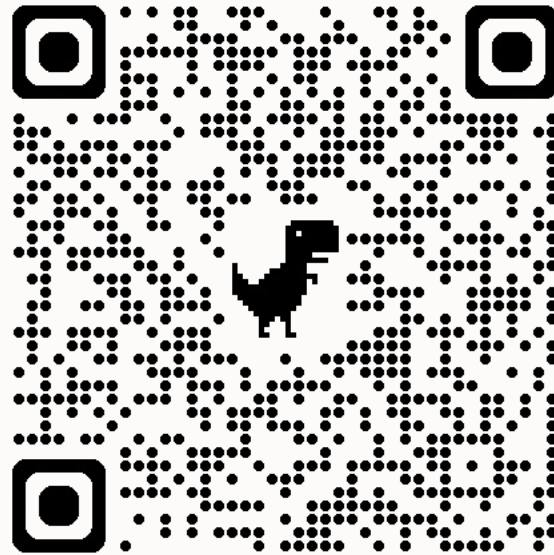
```
$ dbaascli dataguard registerStandby --dbname <value> --standbyDBUniqueName <value> 1...
```

Deregister Standby

```
$ dbaascli dataguard deregisterStandby --dbname <value> --standbyDBUniqueName <value> 1...
```

¹... Not all parameters listed. Please check the command definition

Stay tuned



Operations Advisory
GitHub Repository



Thank you

ORACLE